Consume a Fabric data agent with the Python client SDK (preview)

This article shows how to use the Python client SDK to add a Fabric data agent to web apps and other clients by using interactive browser authentication. You sign in through a browser with your Microsoft Entra ID credentials, and the data agent runs with your permissions. Adding the data agent to external apps lets you build custom interfaces, embed insights in existing workflows, automate reports, and let users run natural language data queries. This approach gives you data agent capabilities while you keep full control of the user experience and app architecture.

> **Important**
>
> This feature is in **preview**.

Prerequisites

- A paid F2 or higher Fabric capacity, or a Power BI Premium per capacity (P1 or higher) capacity with Microsoft Fabric enabled
- Fabric data agent tenant settings is enabled.
- Cross-geo processing for AI is enabled.
- Cross-geo storing for AI is enabled.
- At least one of these, with data: A warehouse, a lakehouse, one or more Power BI semantic models, a KQL database, or an ontology.
- Power BI semantic models via XMLA endpoints tenant switch is enabled for Power BI semantic model data sources.

Set up your environment in VS Code

1. Clone or download the Fabric Data Agent External Client repository , then open it in VS Code and run the sample client.

2. Create and activate a Python virtual environment (recommended), then install the required dependencies.

   Bash

   ```
   python -m venv .venv
   ```

3. Activate the virtual environment.

Windows

Windows Command Prompt

```
.venv\Scripts\activate
```

Install dependencies

Run this command to install dependencies:

Bash

```
pip install -r requirements.txt
```

> **Note**
>
> - The `azure-identity` package included in `requirements.txt` lets you authenticate with Microsoft Entra ID.
> - `InteractiveBrowserCredential` from the `azure-identity` package opens a browser so the user can sign in with a Microsoft Entra ID account. Use it for local development or apps that allow interactive sign-in.

Configure the client

Choose one of these methods to set the required values (`TENANT_ID` and `DATA_AGENT_URL`):

Environment variables

Set the values in your shell. Replace the placeholder text in angle brackets with your own values.

terminal

```
export TENANT_ID=<your-azure-tenant-id>
export DATA_AGENT_URL=<your-fabric-data-agent-url>
```

See the documentation to find the published data agent URL. Follow the instructions to locate your tenant ID.

Authenticate

Use the `InteractiveBrowserCredential` class to authenticate with Microsoft Entra ID in a browser.

Python

```
from azure.identity import InteractiveBrowserCredential
from fabric_data_agent_client import FabricDataAgentClient
credential = InteractiveBrowserCredential()
```

Create the data agent client

Python

```
client = FabricDataAgentClient(credential=credential)
```

> **Note**
>
> - The `fabric-data-agent-client` package provides the client SDK for connecting to the Fabric data agent.
> - The Python client uses interactive browser authentication: when you run the script, your default browser opens so you sign in to the tenant that hosts the Fabric data agent.

Ask the data agent a question

After you authenticate, interact with the data agent by using the Python client.

Python

```
response = client.ask("What were the total sales last quarter?")
print(f"Response: {response}")
```

The `client.ask` method sends your question to the data agent and returns an object with the answer. You can view the steps the data agent performed and the corresponding queries it generated to get the answer.

Python

```
run_details = client.get_run_details("What were the total sales last quarter?")
messages = run_details.get('messages', {}).get('data', [])
assistant_messages = [msg for msg in messages if msg.get('role') == 'assistant']

print("Answer:", assistant_messages[-1])
```

Optional: Inspect the steps and corresponding query

Inspect the steps the data agent took to arrive at the answer, including any errors during execution.

Python

```
for step in run_details['run_steps']['data']:
        tool_name = "N/A"
        if 'step_details' in step and step['step_details'] and 'tool_calls' in
step['step_details']:
            tool_calls = step['step_details']['tool_calls']
            if tool_calls and len(tool_calls) > 0 and 'function' in tool_calls[0]:
                tool_name = tool_calls[0]['function'].get('name', 'N/A')
        print(f"Step ID: {step.get('id')}, Type: {step.get('type')}, Status:
{step.get('status')}, Tool Name: {tool_name}")
        if 'error' in step:
            print(f"  Error: {step['error']}")
```

This output helps you understand how the agent produced its response and gives transparency when you work with your data in the Python client.

Related content

- Fabric data agent concepts
- Fabric data agent SDK
- Data agent end-to-end tutorial

Last updated on 10/07/2025